

# Bedingte und iterative Tasks (Run If, For Each)

Section 4 · Working with Lakeflow Jobs · Quelle: Kurs 2, Kapitel 8–10

---

Nicht jeder Workflow lässt sich als starre Abfolge von Tasks abbilden. Manchmal soll ein Task nur laufen, wenn ein bestimmtes Ergebnis vorliegt; manchmal muss dieselbe Logik auf eine unbekannte Anzahl von Eingabewerten angewendet werden. Für solche Fälle bietet Lakeflow Jobs drei fortgeschrittene Task-Muster: **Run-if-Abhängigkeiten**, **If/Else-Tasks** und **For-Each-Tasks**. Zusammen erlauben sie dynamische Workflows, die auf Laufzeitbedingungen reagieren, statt starr vorgegeben zu sein.

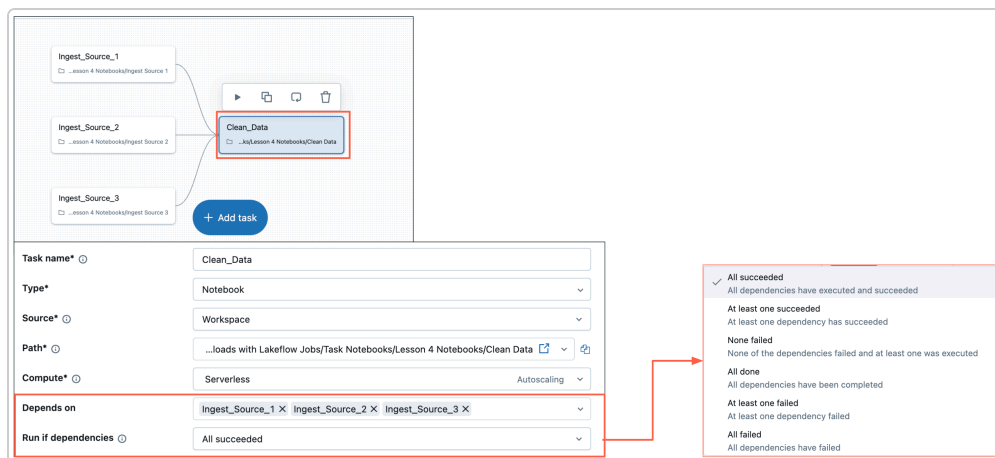
## 1. Run-if-Abhängigkeiten (Run If Conditions)

Standardmäßig wartet ein Task, bis alle vorgelagerten Tasks erfolgreich abgeschlossen sind („All succeeded“). Für robustere Workflows lässt sich diese Bedingung jedoch feiner steuern:

- **All succeeded** – alle Vorgänger-Tasks müssen erfolgreich sein (Standardverhalten).
- **At least one succeeded** – sinnvoll bei redundanten Datenquellen oder Verarbeitungspfaden; der Workflow läuft weiter, sobald mindestens ein Vorgänger erfolgreich war.
- **None failed** – erlaubt Ausführung auch dann, wenn Vorgänger-Tasks übersprungen wurden, solange keiner davon explizit fehlgeschlagen ist.
- **All done** – läuft, sobald alle Vorgänger-Tasks abgeschlossen sind, unabhängig von deren Ergebnis.
- **At least one failed** – läuft, sobald mindestens ein Vorgänger fehlgeschlagen ist – nützlich etwa für Aufräum- oder Benachrichtigungs-Tasks.
- **All failed** – läuft nur, wenn alle Vorgänger-Tasks fehlgeschlagen sind.

Das sind die sechs festen, in der UI/API vorgegebenen Optionen – eine frei definierbare, benutzerdefinierte Bedingung gibt es nicht.

Diese Bedingungen erhöhen die Ausfallsicherheit eines Workflows deutlich: Ist ein Task etwa so konfiguriert, dass er läuft, sobald mindestens einer von zwei Vorgängern erfolgreich war, kann er trotzdem starten, wenn einer der beiden Vorgänger fehlschlägt. Das verhindert Kaskadenausfälle und spiegelt reale Geschäftsprozesse wider, bei denen es oft mehrere gültige Wege zum Ziel gibt.



Unterschiedliche Run-if-Bedingungen werden im Job-DAG durch verschiedene Linienstile visualisiert.

## 2. If/Else-Tasks

Während Run-if-Bedingungen nur den *Status* vorheriger Tasks berücksichtigen, ermöglichen **If/Else-Tasks** echte boolesche Logik basierend auf Daten, Parametern oder berechneten Kennzahlen. Unterstützt werden die Operatoren `==`, `!=`, `>`, `>=`, `<` und `<=`. Typische Anwendungsfälle sind Datenqualitäts-Gates (Verzweigung je nach Datensatzanzahl oder Null-Anteil), unterschiedliche Verarbeitungsstrategien je nach Datenvolumen oder umgebungsabhängige Logik (Dev/Stage/Prod). Jeder der beiden Zweige – **True** und **False** – kann beliebig viele eigene Folge-Tasks enthalten.

Ein konkretes Beispiel aus der Praxis: Ein vorgelagerter Task prüft eine Tabelle auf doppelte Datensätze und speichert das Ergebnis als Task Value.

```
# Im vorgelagerten Task: Duplikate pruefen und Ergebnis als Task Value speichern
df = spark.sql("SELECT * FROM customers_sales_silver")
duplicate_exists = df.count() > df.dropDuplicates().count()
dbutils.jobs.taskValues.set(key="has_duplicates", value=duplicate_exists)
```

Der If/Else-Task greift anschließend über eine dynamische Wertreferenz auf dieses Ergebnis zu und prüft die Bedingung `tasks.customers_sales_summary.values.has_duplicates == true`. Ist die Bedingung wahr, läuft der Zweig, der Duplikate bereinigt; andernfalls läuft direkt die Transformation ohne Bereinigungsschritt.

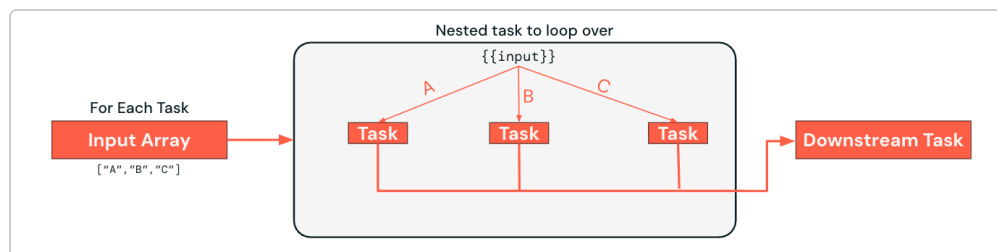
## 3. For-Each-Tasks: iterative Verarbeitung

Ein **For-Each-Task** wiederholt dieselbe Logik für jedes Element einer Eingabeliste – etwa für jeden Bundesstaat, jede Kundengruppe oder jede zu verarbeitende Datei. Er besteht aus zwei Bausteinen:

- **Der For-Each-Container** – verwaltet die Iterationslogik, die Eingabeliste ( `Inputs` ) und die Nebenläufigkeit ( `Concurrency` ), also wie viele Iterationen parallel laufen dürfen.
- **Der verschachtelte (nested) Task** – die eigentliche Arbeit, die pro Iteration ausgeführt wird. Er kann ein beliebiger Task-Typ sein (Notebook, SQL, Python) und erhält das jeweilige Listenelement über die Referenz `{{input}}` .

Downstream-Tasks hängen dabei nur vom gesamten For-Each-Container ab, nicht von jeder einzelnen Iteration – das hält den DAG übersichtlich, auch wenn im Hintergrund Dutzende Iterationen laufen. Ein Beispiel: Aus einer Tabelle mit Bestelldaten sollen separate Auswertungstabellen für die Bundesstaaten Kalifornien, New York und Virginia erzeugt werden, jeweils mit demselben Notebook, aber unterschiedlichem Parameterwert:

```
# Konfiguration des For-Each-Containers (vereinfachtes Beispiel)
# Inputs:      ["CA", "NY", "VA"]
# Nested Task: Notebook "9.5 - For Each: Customer orders State"
# Parameter:   state = {{input}} -> wird pro Iteration automatisch befüllt
```



*Der For-Each-Task durchläuft eine Eingabeliste und führt für jedes Element den verschachtelten Task aus.*

**Ergänzt aus der offiziellen Databricks-Dokumentation:** Databricks führt diese drei Muster unter dem Oberbegriff „Control the flow of tasks“ zusammen. Run-if-Bedingungen sind dabei **kein eigener Task-Typ**, sondern eine Einstellung an der Abhängigkeit („Depends on“) eines Tasks zu seinen Vorgängern; If/Else-Tasks werden dagegen über einen eigenständigen Task-Typ mit boolescher Ausdrucksauswertung konfiguriert. Für For-Each-Tasks unterstützt die aktuelle API mehrere Methoden zur Parameterübergabe an den verschachtelten Task, nicht nur die einfache `{{input}}` - Referenz.

Quelle: [Control the flow of tasks within Lakeflow Jobs – Databricks-Dokumentation](#)

